Metodología Mejorada: Agentic CI/CD Loop v2

Integrando las fortalezas de tu ejemplo (disciplina operacional) con la naturaleza determinista del Crítico que propuse (comandos con umbral cero, no opiniones), esta es la versión refinada.

Principios Fundacionales

1. Determinismo sobre opinión: Todo baremo debe ser materializable en un comando medible. Si no hay comando, el ítem no existe.
2. Cirugía, no refactor: El Actor no "mejora" código; ejecuta intervenciones quirúrgicas enumeradas, una a una.
3. Regresión antes que corrección: Antes de verificar que el bug se arregló, se verifica que no se rompió nada que funcionaba.
4. No improvisación: Si las precondiciones no se cumplen, el bucle no arranca. No se inventan caminos.
5. Perímetro explícito: Lo no listado es protegido por defecto. Tocar lo protegido es fallo, no mejora.

Roles Refinados

Rol	Responsabilidad	Regla crítica

Orquestador	Traduce el bug en baremo, descompone en intervenciones atómicas, coordina el bucle, escala a humano si se agotan iteraciones.	No escribe código. No opina sobre calidad. Solo aplica el veredicto del Crítico.
Actor	Recibe una intervención atómica, el baremo táctico y el perímetro. Ejecuta solo lo listado.	No decide si terminó. No toca lo protegido. No reescribe lo no pedido.
Crítico	Ejecuta comandos con umbral cero contra el baremo. Emite JSON determinista.	No lee el código para opinar. Corre comandos y compara contra thresholds.

Fases del Bucle

Fase 0 — Precondiciones (NUEVA)

Antes de cualquier cambio, el Orquestador verifica:

1. Snapshot de regresión: Ejecuta el script con el dataset real y guarda la salida actual. Esa copia es el golden master.

2. Respaldo: `git stash` o copia del archivo original.
3. Lista de archivos tocables: Definida explícitamente. Ej: `['scripts/generar_grafos.py', 'tests/test_generar_grafos.py']`.
4. Lista de archivos protegidos: Lo no listado es protegido por defecto. Ej: `algoritmo_grafo.py` marcado como intocable.
5. Ejecutabilidad del entorno: El script corre en el sandbox. Las dependencias están instaladas. Los tests existentes pasan en baseline.

Si cualquiera falla: El bucle no arranca. El Orquestador reporta a humano: *"Precondición X no superada: [motivo]. ¿Autoriza continuar o aborta?"*. No hay improvisación.

Fase 1 — Descomposición Quirúrgica

El Orquestador toma el plan y lo divide en intervenciones atómicas. Cada intervención es:

- Una única modificación, descrita en una frase.
- Validable de forma aislada.
- Con su propio baremo táctico derivado del baremo estratégico.

Ejemplo: en lugar de "arregla el bug y añade tests", se generan dos intervenciones:

- Intervención 1: "Añadir guarda al inicio de `construir_grafo()` que devuelva grafo vacío válido cuando no hay aristas."
- Intervención 2: "Añadir test que cubra el caso de nodo aislado."

Nunca se acumulan cambios sin verificar cada intervención por separado.

Fase 2 — Implementación del Actor

El Actor recibe:

- La intervención atómica.
- El baremo táctico (ítems medibles).
- El perímetro (archivos tocables + protegidos).
- El feedback de iteraciones anteriores (si existe).

El Actor propone un diff. No decide si está terminado; lo entrega al Crítico.

Fase 3 — Gauntlet Determinista del Crítico (REFINADA)

El Crítico no lee el código para opinar. Ejecuta comandos con umbral cero contra cada ítem del baremo táctico. El orden es obligatorio — un ítem fallido en cualquier fase detiene el Gauntlet y emite veredicto negativo sin continuar.

Orden obligatorio:

1. Regresión: Re-ejecuta el script con el dataset real. Compara contra el snapshot. La salida debe ser idéntica salvo el caso corregido.
2. bash
3. `diff <(python scripts/generar_grafos.py dataset.json) snapshot_regresion.txt`
4. `# Umbral cero: diff vacío salvo líneas esperadas`
5. Perímetro de archivos:
6. bash
7. `git diff --name-only HEAD`
8. `# Umbral cero: output == lista tocable permitida`
9. Perímetro de líneas (simplicidad):
10. bash
11. `git diff --stat HEAD`
12. `# Umbral: insertions ≤ N, deletions == 0`
13. Estática:
14. bash
15. `python -m py_compile scripts/generar_grafos.py && pylint scripts/generar_grafos.py`
16. `# Umbral cero: exit 0, sin warnings`

17. Dinámica:

```bash
pytest tests/test_generar_grafos.py -v
# Umbral cero: exit 0, todos los tests pasan
```

21. Lógica (fuzzing del Crítico): El Crítico genera casos extremos y los ejecuta:

```python
# El Crítico corre en el sandbox:
assert construir_grafo([], []) no crashea
assert construir_grafo([nodo_aislado], []) devuelve grafo válido
```

Si cualquier ítem falla, el Crítico emite veredicto negativo con el comando que falló como evidencia, no con opinión.

Fase 4 — Veredicto Estructurado (CONSTRAINED)

El Crítico emite un JSON con schema obligatorio. No puede divagar:

```json
{
 "aprobado": false,
 "items_fallados": ["perimetro_lineas", "regresion"],
 "comandos_fallidos": [
   {
     "item": "perimetro_lineas",
     "comando": "git diff --stat HEAD",
     "esperado": "≤ 10 insertions, 0 deletions",
     "obtenido": "47 insertions, 12 deletions",
     "evidencia": "líneas 45-58 reordenadas, algoritmo protegido tocado"
   },
   {
```

```
      "item": "regresion",
      "comando": "diff salida snapshot_regresion.txt",
      "esperado": "vacío salvo caso corregido",
      "obtenido": "12 líneas modificadas no esperadas",
      "evidencia": "líneas 45-58 alteradas"
    }
  ],
  "vuelve_a": "Actor",
  "iteracion_actual": 1,
  "max_iteraciones": 5
}
```

Fase 5 — Auto-Corrección del Actor

El Orquestador toma `comandos_fallidos` y se lo devuelve al Actor con el feedback estructurado. El Actor no ve "opiniones del Crítico"; ve comandos ejecutados, esperado vs. obtenido, y evidencia `archivo:línea`. Reescribe el diff.

Fase 6 — Cierre Autónomo

Cuando el Crítico emite `aprobado: true` para una intervención, el Orquestador:

1. Actualiza el snapshot de regresión (la salida correcta se convierte en el nuevo golden master).
2. Pasa a la siguiente intervención atómica.
3. Cuando todas las intervenciones pasan, hace commit con mensaje auto-generado y entrega al usuario.

Las Tres Capas Integradas

Capa	Qué hace	Cómo se materializa
Capa 1: Reglas escritas	Premisas que el Orquestador aplica al redactar el plan y el Actor aplica al implementar.	System Prompt con la lista de archivos tocables y protegidos. La regla "no tocar lo no listado" opera antes de escribir código.
Capa 2: Verificación estructurada	El Crítico emite JSON con campos fijos, no prosa.	Schema obligatorio. `aprobado` es booleano, no opinión. `comandos_fallidos` es array de evidencia, no párrafo.
Capa 3: Intercepción programática	Comandos con umbral cero que el sandbox corre antes de cualquier entrega.	`git diff --name-only` debe ser exactamente la lista permitida. `grep` de residuos da cero. `pytest` termina en `exit 0`. Si cualquiera falla, la entrega no sale del sandbox.

Lógica de Control y Salvaguardas

python

```
def agentic_cicd_loop(bug_report):
   # Fase 0: Precondiciones
   snapshot = ejecutar_y_guardar_salida_actual()
   if not verificar_entorno() or not respaldar():
       return escalar_a_humano("Precondición fallida")

   tocables = definir_tocables()
   protegidos = definir_protegidos()

   # Fase 1: Descomposición
   intervenciones = orquestador.descomponer(bug_report)

   for interv in intervenciones:
       baremo_tactico = orquestador.derivar_baremo(interv)
       iteracion = 0
       aprobado = False

       while not aprobado and iteracion < MAX_ITER:
           iteracion += 1

           # Fase 2: Actor implementa
           diff = actor.ejecutar_intervencion(interv, baremo_tactico,
feedback_previo)

           # Fase 3-4: Crítico ejecuta Gauntlet determinista
```

```
          veredicto = critico.ejecutar_gauntlet(
              diff=diff,
              baremo=baremo_tactico,
              tocables=tocables,
              protegidos=protegidos,
              snapshot=snapshot,
              orden=["regresion", "perimetro_archivos",
"perimetro_lineas",
                     "estatica", "dinamica", "logica"]
          )

          if veredicto.aprobado:
              aprobado = True
              snapshot = actualizar_snapshot()  # nuevo golden master
          else:
              feedback_previo = veredicto.comandos_fallidos

      if not aprobado:
          return escalar_a_humano(f"Intervención {interv.id} agotó
iteraciones")

   commit_autogenerado()
   return entregar_a_usuario()
```

Diferencias Frente a la Versión Anterior

Aspecto	v1 (mi propuesta original)	v2 (esta versión)
Precondiciones	Ninguna. Arrancaba directo.	Fase 0 obligatoria: snapshot, respaldo, perímetro, entorno.
Granularidad	El Actor corregía el bug completo.	Intervenciones atómicas, validadas una a una.
Orden del Gauntlet	Sin orden estricto.	Orden obligatorio: regresión primero, siempre.
Naturaleza del Crítico	LLM que lee y juzga el código.	Executor de comandos con umbral cero. No opina, mide.

Veredicto	JSON libre.	Schema obligatorio con `comandos_fallidos` (evidencia comando + esperado + obtenido).
Perímetro	Asumido.	Explícito: tocables + protegidos. Lo no listado es protegido.
Regresión	Mencionada como "un test más".	Golden master obligatorio, validado antes que nada.
Tres capas	Mencionadas sueltas.	Integradas con su materialización concreta.

Lo Que Esta Versión Garantiza

- El Crítico no puede ser convencido. No lee código para juzgar; corre comandos y compara. Un LLM persuasivo no convence a un `git diff --name-only`.
- El Actor no se pasa de listo. Tocar lo protegido genera un `exit 1` en el comando de perímetro, no una opinión encontrada en revisión.

- No se acumulan fallos. Cada intervención se valida aislada, sin arrastrar defectos de una a la siguiente.
- No se rompe lo que funcionaba. La regresión se verifica antes que la corrección. El bug se arregla sin collateral damage.
- No hay improvisación. Si el entorno no está listo, no se arranca. Si se agotan iteraciones, se escala. El sistema no inventa caminos.

Esta es la versión que yo usaría en producción. La fusión de tu disciplina operacional con el Crítico determinista convierte el Agentic CI/CD Loop de "un sistema que se corrige solo" en "un sistema que se corrige solo sin traicionar el baremo".